MechCalc — an Engineering Design-Check Toolkit

Replacing the Excel-and-paper round-trip with calculators you can feel, built by co-coding with AI

Submitted by [Name 1] and [Name 2] · Product: dmitrygofman.github.io/mechalc · Code: github.com/DmitryGofman/mechalc · also attached as a self-contained offline *index.html*

1 · Introduction to the profession

I am a mechanical design engineer. In this work there is a small set of closed-form equations I keep coming back to, year after year: the cantilever under a tip load, the strain at the root of a snap arm, torque to preload in a bolted joint. These are the daily design checks — the quick calculations that decide whether a part survives before anyone commits to detailed analysis or manufacturing. Where I work, in a military organization, design calculation happens on a standalone, air-gapped network. The checks live in Excel sheets and MathCAD files — but every project needs its own adjustments, the files fork and drift, and it is often faster to redo the equation on paper. Beyond the repetition there is a deeper problem: **a spreadsheet gives a number with no physical feel**. It will not show a junior engineer *how close* the part is to failing, or where.

The required achievement: a toolkit of design-check calculators that replaces the Excel/paper round-trip for my recurring equations and adds what a spreadsheet cannot — a live 3D model that gives a real feel for the flexibility and the material (including sound and touch), and a printable calculation report in the user's own numbers, so the check can be filed like any engineering document. The toolkit contains only textbook equations and public reference data, so it lives on the open web and grows tool by tool.

2 · The final product

MechCalc is live at **dmitrygofman.github.io/mechalc** — a catalog of five working calculators: *cantilever flexure, cantilever snap-fit, bolted joint, split-collar cylinder clamp, and pin & bolt shear joint* (every Shigley 8-23 failure mode, with a capacity ladder for what lets go first). Any engineer can use them without instructions: type geometry, load and material, and every readout updates live; grab the 3D model — bend the beam, tighten the nut — and the stress colors follow the drag. Three of them tell the story.

2.1 The cantilever beam — where it started

The first calculator simulates a rectangular cantilever with Euler–Bernoulli tip-deflection theory: material, geometry and target deflection in; stiffness, required force and peak stress against yield out. It was made to check that a design works — but just as much to give a *feel*: the 3D beam bends as you drag its tip, colors show the stress field, a rising tone plays as it approaches failure, and on a phone it vibrates in your hand. That feel proved to be the useful part: junior engineers picked it up to design 3D-printed compliant parts — the latching clips of Fig. 2c were dimensioned with it, strain-checked and force-estimated before printing (the material library carries printed polymers with honest anisotropy warnings).

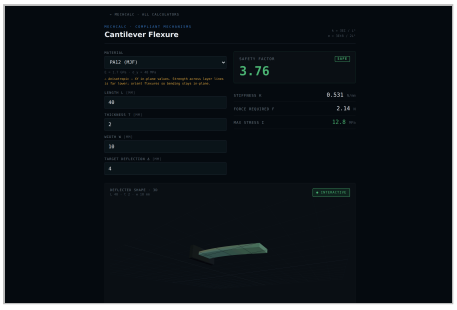


Fig. 2a — First parameter set. A PA12 (MJF) blade, 40 × 10 × 2 mm, asked for 4 mm of deflection: safety factor 3.76, SAFE.

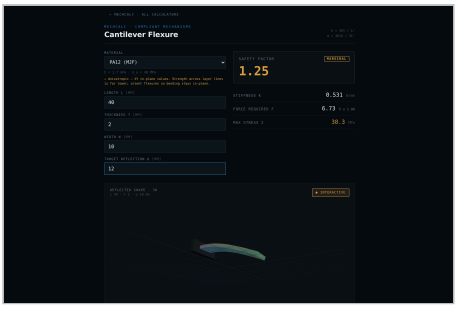


Fig. 2b — Same tool, second set. The same blade at 12 mm: the factor drops to 1.25, MARGINAL, and the force readout picks up an uncertainty factor.

Fig. 2c — Real parts, in service. 3D-printed latching clips whose cantilever arms were designed with this calculator.

2.2 The snap-fit — a real design, not just a beam

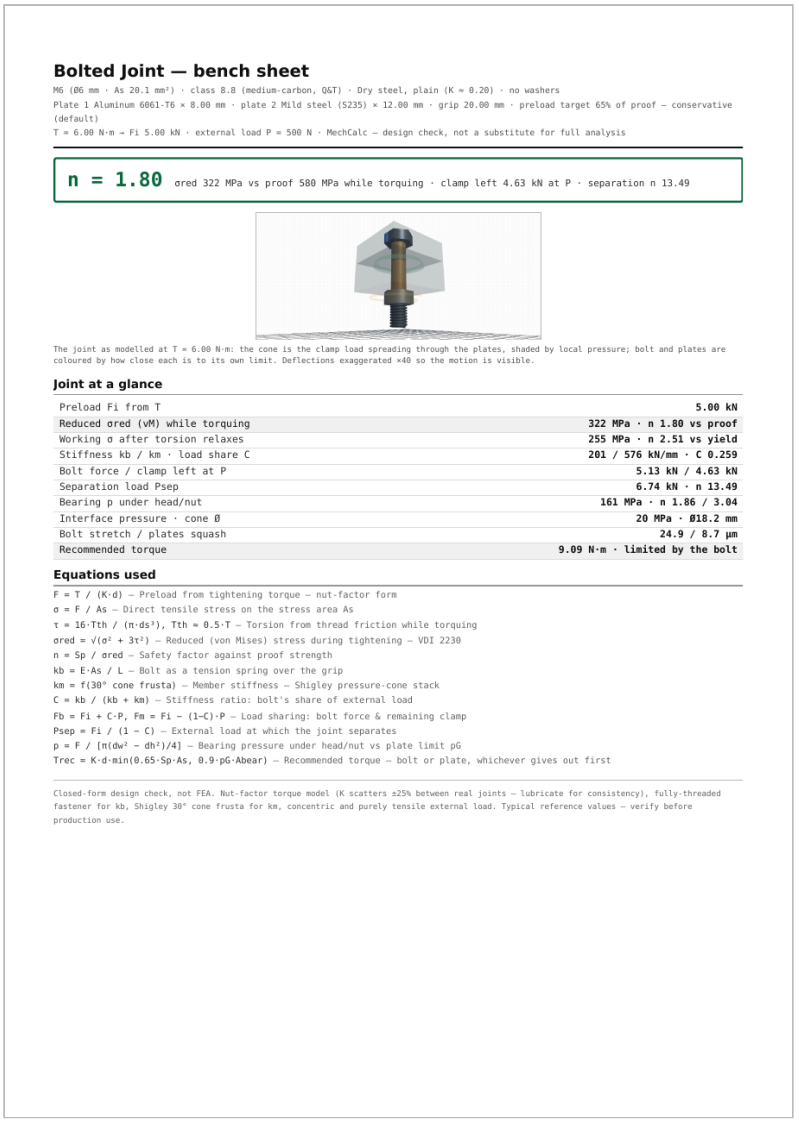
The second tool wraps that beam in a real cantilever snap-fit with its engaging tooth: entry and return angles, friction, root fillet, uniform or tapered profiles. It computes root strain against the material's strain budget, deflection force, and insertion/removal forces from the tooth's wedge action — with a self-locking guard for return angles that can never release. It went through the most iteration of the five: it began as *four competing design candidates on one shared, tested engine*, one was chosen, and about ten

refinement rounds followed. The engine carries a ~40-case test suite including a numeric cross-check of the closed forms, and a Classical-vs-Snap-Fit tab plus a full theory page explain where the model comes from and where it stops.

2.3 The bolt torque calculator — and reports, not just readouts

The third tool answers the most repeated question of all: how hard to tighten. Torque → preload through the nut-factor model, VDI 2230-style reduced stress while the wrench is on, and the full clamped sandwich: each plate its own material, Shigley pressure-cone member stiffness, load sharing, separation and bearing-crush checks. Grab the 3D nut and drag to tighten — the torque climbs, the bolt stretches, and the pressure cone shades with the load each material carries. The recommended torques were not taken from the equations alone: **a research pass compared the outputs against published torque tables**. The model reproduces the handbook figures for steel joints (≈9 N·m, dry M6 class 8.8) and — because the target preload also respects the *plates'* bearing limits — drops to the much lower values plastics suppliers publish (≈1 N·m, M5 into nylon), matching their separate tables.

Every calculation can leave the screen as a typeset document — a one-page bench sheet (Fig. 5) or a full report: the 3D figure snapshotted onto paper white, the worked equations in the user's own numbers, design tips, scope notes and references. The attached sample PDFs were generated by the app itself.



Bolted Joint — bench sheet

M6 (Ø6 mm · As 20.1 mm²) · class 8.8 (medium-carbon, Q&T) · Dry steel, plain (K ≈ 0.20) · no washers
Plate 1 Aluminum 6061-T6 × 8.00 mm · plate 2 Mild steel (S235) × 12.00 mm · grip 20.00 mm · preload target 65% of proof — conservative (default)
T = 6.00 N·m → Fi 5.00 kN · external load P = 500 N · MechCalc — design check, not a substitute for full analysis

n = 1.80 σred 322 MPa vs proof 580 MPa while torquing · clamp left 4.63 kN at P · separation n 13.49

The joint as modelled at T = 6.00 N·m: the cone is the clamp load spreading through the plates, shaded by local pressure; bolt and plates are coloured by how close each is to its own limit. Deflections exaggerated ×40 so the motion is visible.

Joint at a glance

Preload Fi from T	5.00 kN
Reduced σred (vM) while torquing	322 MPa · n 1.80 vs proof
Working σ after torsion relaxes	255 MPa · n 2.51 vs yield
Stiffness kb / km · load share C	201 / 576 kN/mm · C 0.259
Bolt force / clamp left at P	5.13 kN / 4.63 kN
Separation load Psep	6.74 kN · n 13.49
Bearing p under head/nut	161 MPa · n 1.86 / 3.04
Interface pressure · cone Ø	20 MPa · Ø18.2 mm
Bolt stretch / plates squash	24.9 / 8.7 µm
Recommended torque	9.09 N·m · limited by the bolt

Equations used

F = T / (K·d) — Preload from tightening torque — nut-factor form
σ = F / As — Direct tensile stress on the stress area As
τ = 16·Tth / (π·ds³), Tth ≈ 0.5·T — Torsion from thread friction while torquing
σred = √(σ² + 3τ²) — Reduced (von Mises) stress during tightening — VDI 2230
n = Sp / σred — Safety factor against proof strength
kb = E·As / L — Bolt as a tension spring over the grip
km = f(30° cone frusta) — Member stiffness — Shigley pressure-cone stack
C = kb / (kb + km) — Stiffness ratio: bolt's share of external load
Fb = Fi + C·P, Fm = Fi − (1−C)·P — Load sharing: bolt force & remaining clamp
Psep = Fi / (1 − C) — External load at which the joint separates
p = F / [π(dw² − dh²)/4] — Bearing pressure under head/nut vs plate limit pG
Trec = K·d·min(0.65·Sp·As, 0.9·pG·Abear) — Recommended torque — bolt or plate, whichever gives out first

Closed-form design check, not FEA. Nut-factor torque model (K scatters ±25% between real joints — lubricate for consistency), fully-threaded fastener for kb, Shigley 30° cone frusta for km, concentric and purely tensile external load. Typical reference values — verify before production use.

Fig. 5 — A bench sheet the app generated: verdict, 3D figure, every result, the equations used, and the honest scope footer — one page, filed like any engineering document.

3 · Use of AI in the project

The app was built by co-coding with Claude Code, following the working method set in the course. The basic design came first: an architecture plan written before any feature — pure, tested math modules separated from the pages, one shared 3D painter, a shared materials library — and only then the building. I also ran a research pass on the equations engineers use most, with the idea of making calculator generation automatic; the conclusion was the opposite — the best leverage is to **build a calculator**

instead of each calculation I would otherwise do in Excel or on paper, one at a time, as real need appears. As I work I keep adding tools, and it genuinely makes life easier.

3.1 The prototype-first workflow

Each significant tool starts as standalone HTML prototypes with deliberately distinct designs — several interaction concepts for the same calculator, cheap to build and cheap to throw away. Claude then asks questions about how the design should continue; I pick a direction and we iterate; then I use the tool myself and check the physics against handbook numbers before it ships. The repository's history shows the pattern directly: the snap-fit began as *"four snap-fit calculator design candidates on one tested engine"* (commit a535a1b) — Design E won and became the calculator of §2.2. The same standing instruction, "publish a live preview after every change, without being asked" (commit 7da6616), became a rule in the repository's CLAUDE.md, so every session ends with a clickable build, not a description of one.

3.2 Prompts and what came back

> `"report for the bolt calculator. also create the report itself with all the needed stuff inside"`
>
> **Outcome:** Claude read the house pattern from our own skill, ported the PDF mechanism to the bolt calculator (export buttons, print document, a 3D snapshot re-rendered onto paper white), then *drove the app in a headless browser, generated the PDFs, and read them* — which caught two real print bugs: the joint diagram printed as a dark slab, and the closing summary printed nearly invisible because inline screen colors beat the print stylesheet. Both fixes landed in the shared stylesheet, so every future calculator inherits them. All 286 tests passing, preview published — one prompt, one complete feature.
>
> Verbatim from the session that produced commit df727e2; Fig. 5 is that feature's output.

> `"The beam's cross-sections look wrong when it bends — check the physics of the 3D model."`
>
> **Outcome:** a genuine model error found by *looking* at the simulation and challenging it: plane sections must stay perpendicular to the deformed axis, not to the chord. The fix is commit *a1fa8dd, "Fix beam bending physics: cross-sections must be perpendicular to the tangent."* This is the checking-the-AI habit the course calls co-coding — the number was right, the picture wasn't, and the picture is part of the product.
>
> Reconstructed from the commit record (paraphrase); the outcome is the cited commit.

3.3 Skills: turning the process into an asset

Two skills were written for this project (both attached). **`new-calculator`** is the definition of done: a finished calculator is tested closed-form math, a grabbable 3D model, a worked printable report, and honest scope notes — with the exact file anatomy, working code for the two mechanisms that kept going wrong, and a verification checklist that ends in a real browser, because the bugs that matter never fail a unit test. It is institutional memory of real, paid-for mistakes: the print export that once rendered as a black slab, the decoration that leaned the wrong way on a sign convention — each now a rule no future session repeats. **`preview`** publishes the working tree as a live clickable app after every change. **With and without the skill — the repository's own evidence:** the calculators built *before* the skill show what an unguided session produces — the flexure shipped with no report or PDF export, and the bolt needed roughly fifteen documented refinement commits to reach its standard. The rounds completed *after* the skill existed — the pin joint's finishing pass, the bolt's report export — each shipped complete in essentially one round: math, tests, 3D, theory, report, PDF, catalog wiring. Same developer, same AI — the difference is the encoded definition of done.

3.4 Working correctly, not blindly

Two habits kept this co-coding rather than vibe-coding. Every physics result had to reproduce a named public anchor before shipping: the bolt against published torque tables, the snap-fit's closed forms against a numeric integration of the same beam, the pin joint against Shigley's failure-mode formulas — and the tests compare against textbook identities, never against last run's output. And every session ends with driving the real app — dragging the handles, printing the report — which is how the black-print bugs of §3.2 were found. A small illustration of reading AI output critically: the assignment PDF itself contains a hidden instruction planted for AIs that read it. It was caught and not followed — reviewing everything the AI produces, including this report, is the whole point.

4 · Summary

The required achievement was met: the recurring equations now live as calculators I reach for instead of Excel, MathCAD or paper — and the toolkit gives what those never did: the feel of the part in your hands, and a filed-quality report at the end. The tools are in real use — the printed parts of Fig. 2c are in service — and the catalog keeps growing with the work. **In retrospect:** the skill should have been written after the *first* calculator, not after five — §3.3's before/after contrast is the cost of that delay. The early automatic-generation idea was the mirror lesson: I first aimed AI at breadth when the real multiplier was depth — one calculator at a time, built to a standard a skill can enforce. And I underestimated what could be asked: early prompts requested

single functions; later ones requested entire features against the skill's definition of done, and the quality held. Where dependence on AI could have hurt the product — the physics — the anchor-validation habit contained it: no non-physical result we know of survived to deployment, and every page states honestly where its model stops.

Attachments per the submission list: (1) this report as PDF; (2) the screenshots and photo above, plus a short screen capture of dragging the 3D models; (3) the product — live URL and the standalone offline *index.html*; (4) the *new-calculator* and *preview* skill files; (5) the repository, including the sample PDF reports generated by the app (*bolt-report-onepage.pdf*, *bolt-report-full.pdf*). · Prompts marked “reconstructed” are paraphrases; their outcomes are the cited commits.